PROJECT BLUEPRINT

Beyond Accuracy: A Calibration- and Explainability-Aware Benchmark of CNN, ViT, and Hybrid Architectures for Brain Tumor MRI Classification

A complete technical replication guide for building a reproducible, fair architectural comparison pipeline for four-class brain tumor MRI classification.

This document contains NO results — only precise technical specifications required to replicate the entire project from scratch.

Table of Contents

1. Project Overview & Objective

2. Dataset Acquisition & Preparation

   2.1 Main Dataset (Kaggle)

   2.2 External Dataset (Figshare)

   2.3 Data Splitting Strategy

   2.4 Provenance & Overlap Audit

3. Model Architectures

   3.1 ResNet-50 (CNN Baseline)

   3.2 EfficientNet-B0 (Lightweight CNN Baseline)

   3.3 ViT-B/16 (Vision Transformer Baseline)

   3.4 Hybrid CNN-ViT (Combined Architecture)

4. Data Augmentation Pipeline

5. Training Protocol & Hyperparameters

6. Evaluation Metrics to Compute

6.1 Classification Metrics

6.2 Calibration Metrics

6.3 Explainability Metrics

6.4 Efficiency Metrics

6.5 Statistical Significance Testing

7. Explainability Analysis (Architecture-Specific XAI)

8. Experiment Execution Plan

9. Output Deliverables

10. Known Limitations & Caveats

1. Project Overview & Objective

This project builds a reproducible benchmark pipeline that performs a fair, head-to-head comparison of four deep learning architecture families — Convolutional Neural Networks (CNNs), a lightweight CNN, Vision Transformers (ViTs), and a CNN-Transformer Hybrid — on the task of four-class brain tumor classification from MRI images.

The core principle is fairness: all four models must be trained under identical conditions (same optimizer, learning rate, scheduler, stopping criterion, data augmentation, and hyperparameters) so that any performance differences are attributable solely to architectural inductive biases, not tuning advantages.

Contribution Statement: The primary contribution of this work is NOT a novel architecture, but rather a rigorous, controlled benchmarking methodology. While CNN-ViT hybrids have been explored in prior work (e.g., R50-ViT, ResViT), existing studies typically use architecture-specific hyperparameter tuning, report only accuracy, and lack calibration or quantitative explainability analysis. This project provides: (1) a strictly controlled comparison under identical training conditions, (2) evaluation beyond accuracy — including calibration (ECE) and quantitative explainability (deletion faithfulness, perturbation stability), (3) multi-seed reporting for statistical reliability, and (4) a transparent provenance audit of the evaluation data.

Key Design Principles

- Identical training protocol across all architectures — no model-specific hyperparameter tuning.
- Four architectures (ResNet-50, EfficientNet-B0, ViT-B/16, Hybrid CNN-ViT) for comprehensive coverage.

- Three random seeds (42, 1337, 2026) for every experiment to report mean ± std.
- Evaluation goes beyond accuracy: calibration (ECE), explainability (deletion faithfulness, perturbation stability), efficiency (params, FLOPs, inference time), and AUROC are all measured.
- Statistical significance testing (paired t-test / Wilcoxon signed-rank) between model pairs.
- An external dataset (Figshare) is used as a sensitivity check for cross-dataset behaviour.
- Full reproducibility via deterministic seeds, data manifests, and hash audit logs.

Classification Task

Four-class classification of brain MRI slices into: Glioma, Meningioma, No Tumor, and Pituitary.

2. Dataset Acquisition & Preparation

2.1 Main Dataset (Kaggle)

Source: The Kaggle Brain Tumor MRI dataset. This dataset comes with a pre-defined train/test split. The test split is held out exclusively for final evaluation and must never be used during training or validation.

Dataset structure (per class):

Split	Glioma	Meningioma	No Tumor	Pituitary	Total
Training (full)	1,400	1,400	1,400	1,400	5,600
Training subset (~85%)	~1,190	~1,190	~1,190	~1,190	~4,760
Validation subset (~15%)	~210	~210	~210	~210	~840
Testing (held-out)	400	400	400	400	1,600
GRAND TOTAL					7,200

Note: Training/validation subset counts are approximate. Stratified splitting may produce slightly different per-class counts due to rounding. Report actual counts after splitting.

2.2 External Dataset (Figshare)

Source: The official Figshare brain tumor MRI dataset. Originally distributed as MATLAB .mat files, these must be converted to PNG images using the pipeline's "prepare-figshare" command (or equivalent conversion script).

This dataset contains only THREE tumor classes (no "No Tumor" class). Labels are preserved as-is from the original source.

Class	Images
Glioma	1,426
Meningioma	708
Pituitary	930
Total	3,064

2.3 Data Splitting Strategy

1. Step 1: Use the Kaggle dataset's pre-defined test split (1,600 images) as the held-out test set. Do NOT modify this split.
2. Step 2: From the remaining training data (5,600 images), create a validation subset by holding out 15% using a fixed random seed.
3. Step 3: The remaining ~85% becomes the actual training subset (~4,760 images).
4. Step 4: The split must be stratified to maintain class balance.
5. Step 5: Use the same fixed seed for the split across all experiments to ensure identical data partitions.
6. Step 6: After splitting, record and report the exact per-class counts for train and validation subsets.

2.4 Provenance & Overlap Audit

IMPORTANT: The Kaggle dataset card lists Figshare as a contributing source. An overlap audit must be performed:

- Compute SHA-256 hashes for all images in both datasets.
- Check for exact duplicate matches (0 exact SHA-256 matches are expected based on the original audit).
- Perform perceptual near-duplicate detection (the original audit found 706 perceptual near-duplicate pairs).
- Because of this overlap, results on the Figshare external dataset must NOT be presented as evidence of independent cross-dataset generalisation. They should be labelled as a "sensitivity check" only.

3. Model Architectures

Four architectures must be implemented. All models use ImageNet-1k pretrained weights from torchvision as initialisation (except for the Hybrid's custom Transformer head, which is trained from scratch). Each model's original classification head is replaced with a custom head for four-class output.

3.1 ResNet-50 (CNN Baseline)

Architecture: Standard 50-layer residual convolutional network.

Feature extraction: Local feature extraction through strided convolution layers with skip connections.

Key property: Prevents gradient degradation via residual/skip connections.

Pretrained weights: ImageNet-1k (torchvision).

Fine-tuning modification: Replace the original classification head (fc layer) with a new head: Linear(2048, 4) with Dropout(0.2).

Inductive bias: Strong local spatial bias — processes nearby pixels together through convolution kernels.

Approximate parameters: ~25.6M total, ~8K trainable (head only) or ~25.6M (full fine-tune).

3.2 EfficientNet-B0 (Lightweight CNN Baseline)

Added as a lightweight CNN baseline to enable efficiency vs. accuracy trade-off analysis, which is clinically relevant for edge deployment in resource-constrained hospital settings.

Architecture: Compound-scaled CNN using mobile inverted bottleneck convolutions (MBConv) with squeeze-and-excitation.

Key property: Achieves strong accuracy with significantly fewer parameters than ResNet-50 through neural architecture search-derived scaling.

Pretrained weights: ImageNet-1k (torchvision).

Fine-tuning modification: Replace the original classifier with: Linear(1280, 4) with Dropout(0.2).

Inductive bias: Local spatial bias (like ResNet) but with depth-wise separable convolutions for efficiency.

Approximate parameters: ~5.3M total.

3.3 ViT-B/16 (Vision Transformer Baseline)

Architecture: Base Vision Transformer with 16×16 patch tokenisation.

Mechanism: Full global self-attention from the first layer — no convolution operations.

Tokenisation: Input image (224×224) is divided into 196 non-overlapping 16×16 pixel patches, each linearly projected into a 768-dimensional token.

Pretrained weights: ImageNet-1k (torchvision).

Fine-tuning modification: Replace the original classifier head with a new MLP head: Linear(768, 4) with Dropout(0.2).

Inductive bias: Minimal spatial bias — learns spatial relationships entirely from data through attention.

Approximate parameters: ~86.6M total.

3.4 Hybrid CNN-ViT (Combined Architecture)

Design rationale: This architecture combines a CNN backbone with a Transformer encoder head. While the general idea of CNN-Transformer hybrids has been explored in prior work (e.g., R50-ViT in Dosovitskiy et al. 2020, ResViT, and various MDPI publications), the specific configuration below is custom-built for this benchmark. The purpose is NOT to claim architectural novelty, but to include a representative hybrid in the controlled comparison alongside pure-CNN and pure-Transformer baselines.

CNN Backbone: ResNet-50 (pretrained, ImageNet-1k) is used as the spatial feature extractor. Extract feature maps from layer4 output (spatial size: 7×7, channels: 2048).

Projection Layer: A learned linear projection (nn.Linear(2048, 512)) reduces each spatial token from 2048 dimensions to 512 dimensions. This bridges the CNN feature dimension to the Transformer embedding dimension.

Tokenisation: The 7×7 = 49 spatial positions from layer4 are flattened into a sequence of 49 tokens, each of dimension 512 (after projection).

Positional Encoding: Learnable positional embeddings of shape (49, 512) are added to the token sequence to preserve spatial arrangement information. Without this, the Transformer would treat tokens as an unordered set.

[CLS] Token: A learnable classification token (dimension 512) is prepended to the 49-token sequence, resulting in a 50-token input to the Transformer. The [CLS] token's output is used for final classification (consistent with standard ViT practice).

Transformer Encoder: A 4-layer, 8-head Transformer encoder with embedding dimension 512, feedforward dimension 2048, and dropout 0.1 processes the 50-token sequence.

Classification Head: The [CLS] token output from the final Transformer layer is passed through Dropout(0.2) → Linear(512, 4) for 4-class classification.

Key design: Combines CNN's local spatial bias (good at extracting low/mid-level features from MRI textures) with Transformer's global reasoning capability (good at capturing long-range spatial dependencies between tumor regions).

Architecture Data Flow Summary: Input (224×224×3) → ResNet-50 layers 1–4 → Feature map (7×7×2048) → Flatten to 49 tokens → Linear projection (2048→512) → Add positional embeddings + prepend [CLS] token → 4-layer Transformer encoder (dim=512, heads=8) → [CLS] output → Dropout(0.2) → Linear(512, 4) → Logits.

4. Data Augmentation Pipeline

CRITICAL: The same augmentation pipeline must be applied identically to all four models. Augmentations are applied only to the training subset. Validation and test sets receive only resize and normalisation.

Training Augmentation

Transform	Parameters	Rationale
Resize	224 × 224 pixels	Match pretrained model input size
RandomHorizontalFlip	p = 0.5	Brain MRI has no canonical left-right orientation in 2D slices
RandomRotation	±15°	Simulate slight acquisition angle variation; kept small to avoid anatomically implausible images
ColorJitter	brightness=0.1, contrast=0.1	Simulate scanner intensity variation; kept subtle for grayscale MRI
ToTensor	—	Convert PIL image to tensor, scale [0, 1]
Normalize	mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]	ImageNet statistics (required for pretrained weights)

Validation / Test Transforms

Transform	Parameters	Rationale
Resize	224 × 224 pixels	Match training input size
ToTensor	—	Convert PIL image to tensor, scale [0, 1]
Normalize	mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]	ImageNet statistics

Design choice: Augmentations are intentionally conservative. Medical images have specific anatomical constraints — aggressive augmentations (large rotations, heavy color distortion, cutout/mixup) can produce anatomically implausible images that may hurt rather than help model learning.

5. Training Protocol & Hyperparameters

CRITICAL: All four models must be trained under the EXACT SAME protocol below. No architecture-specific hyperparameter tuning is allowed. This is the foundation of the fair comparison.

Hyperparameter	**Value**
Optimizer	AdamW
Learning Rate	1×10^{-4} (0.0001)
Weight Decay	1×10^{-2} (0.01)
LR Schedule	Cosine Annealing (T_max = 30, i.e., always based on max epochs regardless of early stopping)
Label Smoothing	0.05
Max Epochs	30
Early Stopping	Patience = 7 epochs, criterion = validation macro F1 score
Batch Size	32
Input Resolution	224 × 224 pixels
Dropout (classification heads)	0.2
Pretrained Initialisation	ImageNet-1k weights from torchvision
Data Augmentation	See Section 4

Note on Weight Decay: Weight decay is set to 0.01 (not 0.0001 as in the original draft). A higher weight decay is standard practice in transformer fine-tuning literature (Dosovitskiy et al. 2020, Touvron et al. 2021) and provides better regularisation. Using the same value across all architectures maintains fairness.

Note on Cosine Annealing: T_max is fixed at 30 (max epochs) regardless of when early stopping triggers. If training stops at epoch 15, the learning rate will have decayed to approximately the midpoint of the cosine schedule. This is acceptable and ensures identical scheduler behaviour across all runs.

Random Seeds

Every model must be trained three times, once with each of the following seeds: 42, 1337, 2026. This means a total of 12 training runs (4 models × 3 seeds). Seeds must control: random weight initialisation of new layers, data shuffling, dropout masks, and any other stochastic operations.

Early Stopping Details

Monitor the validation macro F1 score after each epoch. If the macro F1 does not improve for 7 consecutive epochs, stop training and restore the model weights from the best epoch. Macro F1 is chosen over accuracy because it is robust to class imbalance.

6. Evaluation Metrics to Compute

After training, evaluate each model on the held-out test set (1,600 images). Compute all of the following metrics. Report each as mean ± standard deviation across the 3 seeds.

6.1 Classification Metrics

Accuracy (Acc): Fraction of correctly classified samples out of total test samples.

Macro F1 Score (F1): Harmonic mean of precision and recall, computed per class and then averaged uniformly across all four classes. This is the primary model selection criterion.

Macro Precision (Prec): Mean positive predictive value per class. Measures how many of the model's positive predictions for each class are actually correct.

Macro Recall (Rec): Mean true positive rate per class. Measures clinical sensitivity — the ability to detect each tumor type.

AUROC (AUC): Area under the ROC curve, computed in a one-vs-rest fashion and then macro-averaged. Threshold-independent measure of discriminability.

6.2 Calibration Metric

Expected Calibration Error (ECE): Weighted mean of |confidence − accuracy| across probability bins. Measures how well the model's predicted confidence aligns with its actual accuracy. Lower is better. Critical for clinical trustworthiness. Use 15 equal-width bins.

6.3 Explainability Metrics

Deletion Faithfulness (Del-AUC): Generate saliency maps (Grad-CAM for CNNs, Attention Rollout for ViT) for test images. Progressively delete (zero out) pixels in order of their saliency score (highest saliency first). Plot accuracy vs. fraction of pixels deleted. Compute AUC of this curve. Higher AUC means the saliency map correctly identifies the most important regions (deleting them causes the biggest accuracy drop).

Perturbation Stability (Stab): Add small Gaussian noise perturbations ($\sigma = 0.02$) to the input images. Generate saliency maps for both the original and perturbed inputs. Compute the mean L^2 distance between the two saliency maps. Lower distance means the explanations are more stable/reliable under minor input variations.

6.4 Efficiency Metrics

Report the following for each architecture:

Total Parameters (M): Total number of trainable and frozen parameters in millions.

FLOPs (G): Floating-point operations for a single forward pass (224×224 input) in giga-FLOPs. Use the fvcore or ptflops library for measurement.

Inference Time (ms): Mean inference time per image on the evaluation GPU, measured over the full test set with batch_size=1. Report GPU model used.

6.5 Statistical Significance Testing

With only 3 seeds, standard parametric tests have low power. Apply the following:

- For each pair of models (6 pairs total from 4 models), compute paired differences of macro F1 across the 3 seeds.
- Report the mean difference and standard error.
- Apply the paired t-test (or Wilcoxon signed-rank test as a non-parametric alternative) and report p-values.
- Clearly acknowledge that with n=3 observations, statistical power is limited and results should be interpreted as indicative, not definitive.
- If any two models have overlapping mean ± std ranges, explicitly state that the difference is not statistically significant.

7. Explainability Analysis (Architecture-Specific XAI)

Standard Grad-CAM is designed for CNNs and does not directly apply to Vision Transformers. Each architecture requires an appropriate XAI method.

XAI Method per Architecture

Architecture	XAI Method	Target Layer / Mechanism
ResNet-50	Grad-CAM	Last convolutional layer (layer4[-1].conv3). Gradients of predicted class score w.r.t. feature maps.
EfficientNet-B0	Grad-CAM	Last convolutional layer (features[-1]). Same

		procedure as ResNet-50.
ViT-B/16	Attention Rollout	Multiply attention weight matrices across all 12 encoder layers, average across heads. Extract the [CLS] token's attention over patch tokens to produce a 14×14 saliency map. Upsample to 224×224.
Hybrid CNN-ViT	Grad-CAM (CNN part) + Attention Rollout (Transformer part)	Grad-CAM on ResNet-50 layer4 for CNN-level saliency. Attention Rollout on the 4-layer Transformer head for Transformer-level saliency. Report both visualisations.

XAI Procedure

7. For each model (best seed), select 200 representative test images (50 per class, stratified).
8. Forward pass each image through the model.
9. For CNN models: Compute Grad-CAM on the specified target layer. Weight each feature map channel by the global average of its gradients. Apply ReLU. Upsample to 224×224.
10. For ViT: Compute Attention Rollout across all 12 encoder layers. Extract [CLS]-to-patch attention. Reshape to 14×14 grid. Upsample to 224×224.
11. For Hybrid: Generate both Grad-CAM (CNN backbone) and Attention Rollout (Transformer head) saliency maps.
12. Overlay each saliency heatmap on the original MRI image using a colormap (e.g., jet).

Quantitative Analysis

Beyond visual inspection, compute the two quantitative explainability metrics (Deletion Faithfulness and Perturbation Stability) as described in Section 6.3. Use each architecture's appropriate XAI method (Grad-CAM or Attention Rollout) to generate the saliency maps used in these computations.

8. Experiment Execution Plan

The complete experiment consists of the following sequential phases:

Phase 1: Data Preparation

- Download the Kaggle Brain Tumor MRI dataset (train + test splits).
- Download the Figshare Brain Tumor MRI dataset (.mat files).
- Convert Figshare .mat files to PNG images.
- Verify dataset integrity (file counts, class distributions).

- Split Kaggle training data into ~85% train / ~15% validation (stratified, fixed seed). Record actual counts.
- Perform the provenance/overlap audit (SHA-256 hashing + perceptual duplicate detection).

Phase 2: Model Implementation

- Implement ResNet-50 with custom classification head (Dropout(0.2) → Linear(2048, 4)).
- Implement EfficientNet-B0 with custom classification head (Dropout(0.2) → Linear(1280, 4)).
- Implement ViT-B/16 with custom MLP head (Dropout(0.2) → Linear(768, 4)).
- Implement Hybrid CNN-ViT: ResNet-50 backbone (layer4) → Linear(2048, 512) projection → add learnable positional embeddings → prepend [CLS] token → 4-layer 8-head Transformer encoder (dim=512) → Dropout(0.2) → Linear(512, 4).
- Implement shared training loop with all hyperparameters from Section 5.
- Implement data augmentation pipeline from Section 4.
- Implement early stopping logic (patience=7, criterion=validation macro F1).
- Implement cosine annealing LR scheduler (T_max=30).
- Implement deterministic seed-setting function (covering random, numpy, torch, CUDA).

Phase 3: Training (12 Runs Total)

Execute 12 training runs in total:

- ResNet-50 with seed=42
- ResNet-50 with seed=1337
- ResNet-50 with seed=2026
- EfficientNet-B0 with seed=42
- EfficientNet-B0 with seed=1337
- EfficientNet-B0 with seed=2026
- ViT-B/16 with seed=42
- ViT-B/16 with seed=1337
- ViT-B/16 with seed=2026
- Hybrid CNN-ViT with seed=42
- Hybrid CNN-ViT with seed=1337
- Hybrid CNN-ViT with seed=2026

Each run should save: best model checkpoint (based on validation macro F1), training logs (loss, metrics per epoch), and the epoch at which early stopping triggered.

Phase 4: Evaluation on Internal Test Set

- Load each of the 12 best checkpoints.
- Evaluate on the held-out Kaggle test set (1,600 images).
- Compute per-run: Accuracy, Macro Precision, Macro F1, Macro Recall, AUROC, ECE.

- Aggregate per-model: mean ± std across 3 seeds for each metric.
- Run paired statistical significance tests between all model pairs.

Phase 5: Evaluation on External Dataset (Figshare)

- Load each of the 12 best checkpoints.
- Evaluate on the Figshare dataset (3,064 images, 3 classes only).
- Note: Since Figshare has no "No Tumor" class, evaluation is on 3 classes only.
- Label all external results as "sensitivity check" due to provenance overlap.

Phase 6: Explainability Analysis

- Run architecture-specific XAI (Grad-CAM or Attention Rollout) on 200 test images per architecture (800 total).
- Generate saliency heatmap overlays.
- Compute Deletion Faithfulness (Del-AUC) per model using each model's appropriate XAI method.
- Compute Perturbation Stability (Stab) per model using Gaussian noise perturbations ($\sigma = 0.02$).
- Compare Spearman correlation of stability scores across architectures.

Phase 7: Efficiency Analysis

- Compute total parameter count for each architecture.
- Compute FLOPs for a single forward pass (224×224 input) using fvcore or ptflops.
- Measure mean inference time per image on the evaluation GPU (batch_size=1, over full test set).
- Record GPU model, CUDA version, and PyTorch version for reproducibility.

9. Output Deliverables

The completed project should produce the following artifacts:

Trained Model Checkpoints: 12 checkpoint files (4 models × 3 seeds), each containing the model weights from the best epoch.

Training Logs: Per-epoch training and validation loss, accuracy, and macro F1 for all 12 runs.

Training Curves: Loss and macro F1 vs. epoch plots for all 12 runs, enabling visual comparison of convergence behaviour across architectures.

Per-Run Results Table: A table showing Accuracy, Macro Precision, Macro F1, Macro Recall, AUROC, ECE for each of the 12 individual runs (model × seed).

Aggregated Results Table: A table showing mean ± std of all metrics, grouped by model architecture.

Statistical Significance Table: Paired t-test / Wilcoxon p-values for macro F1 between all model pairs.

Per-Class Breakdown: Precision, recall, and F1 per class per model to identify class-level diagnostic challenges.

Confusion Matrices: One row-normalised confusion matrix per model (best seed or aggregated). Row-normalised means each row sums to 1.0, showing per-class recall at a glance.

Saliency Visualisations: Heatmap overlays for representative test images from each model, using architecture-appropriate XAI methods (Grad-CAM or Attention Rollout).

Explainability Metrics Table: Deletion Faithfulness and Perturbation Stability scores per model.

Efficiency Comparison Table: Total parameters (M), FLOPs (G), and inference time (ms) per model.

External Dataset Results: Performance on Figshare dataset, clearly labelled as sensitivity check.

Data Audit Log: SHA-256 hash lists and perceptual duplicate detection results.

Reproducibility Package: All code, configuration files, random seeds, data manifests, and data augmentation transforms needed for exact replication.

10. Known Limitations & Caveats

The following limitations are inherent to the project design and should be acknowledged:

External dataset provenance overlap: The Kaggle dataset lists Figshare as a contributing source. 706 perceptual near-duplicate pairs exist. External results must be labelled as sensitivity check only, not independent generalisation evidence.

2D slice-level classification: The project classifies individual 2D MRI slices, not 3D volumetric data. This is scoped to match the benchmark dataset format.

Single dataset family: All results are from one dataset family (Kaggle/Figshare). Results may not generalise to other MRI datasets. Three-seed reporting quantifies within-dataset variability.

No architecture-specific tuning: Each model may not be at its individual optimum. This is an intentional design choice to enable fair architectural comparison. A fixed weight decay of 0.01 may not be optimal for CNNs (which sometimes prefer lower values), but this trade-off is necessary for controlled comparison.

ImageNet pretrained initialisation: All models use transfer learning from ImageNet-1k. Results reflect fine-tuning performance, not training-from-scratch capability. This is standard practice in medical imaging.

Batch size constraint: A fixed batch size of 32 is used across all architectures. ViTs sometimes benefit from larger batch sizes, but this constraint is necessary for fair comparison under identical conditions.

Statistical power: With only 3 random seeds, statistical significance tests have limited power (n=3). Results should be interpreted as indicative trends. Increasing to 5+ seeds would strengthen conclusions but increases computational cost.

Hybrid architecture is not novel: The CNN-Transformer hybrid design pattern has been explored in prior work (R50-ViT, ResViT, etc.). The hybrid is included as a representative architecture for the benchmark, not as a novel contribution.

XAI method variation: Different XAI methods are used for different architectures (Grad-CAM for CNNs, Attention Rollout for ViT). Quantitative explainability comparisons across architectures should be interpreted with this caveat — differences may partially reflect method differences, not purely architectural differences.